

District IQ

Chennai Intelligent District Governance Platform

Collector's Unified Application, Web and District Intelligence System

Complete project report — data collection to dashboards, AI and AWS deployment
FarmwiseAI Campus Product Challenge · Task 6 · Chennai District

Prepared 1 October 2026 · Repositories: [ApshanaSP/farmwise](#) (MySQL version) and [ApshanaSP/farmwise-aws](#) (AWS version)

Contents

1. Executive summary
2. Problem statement and goal
3. System architecture at a glance
4. End-to-end workflow (data fetching to the Collector's decision)
5. Stage 1 — Data collection: every source and how it is fetched
6. Stage 2 — News pipeline
7. Stage 3 — district_intel: the curated intelligence layer
8. Stage 4 — Storage: SQLite, MySQL and AWS
9. Stage 5 — The web application (portal and consoles)
10. Stage 6 — Ask District IQ (AI assistant)
11. Stage 7 — AWS deployment and scheduling
12. Complete functionality list
13. Technology stack — what, where, why, and why better
14. AI / ML and analytics methods
15. Security, privacy and access control
16. Quality measures, tests and known limits
17. Development timeline
18. How to run

1. Executive summary

District IQ is a working prototype that gives the Chennai District Collector one place to see what is happening in the district, what needs a decision, and which department is acting on it. It joins **real public data** (news, IMD weather, CPCB air quality, the TN flood-monitoring DSS, AGMARKNET market prices, GCC officials) with **realistic synthetic department records** (citizen grievances, police, PWD, government hospitals) into one curated store, then serves it through role-based web consoles and an AI assistant that never invents numbers.

- **8 source feeds** collected by independent collectors; their fetching code is never modified.
- **district_intel** pipeline: cleans, classifies, geo-locates to the 200 GCC wards, removes duplicates, links the same real-world incident across sources, computes priority, trends, hotspots and briefings (32 tables, 6 views).
- **Citizen portal**: register, file complaints with a map pin and photo, track status.
- **Collector console**: fixed-screen, 4-page dashboard (Overview, Briefing, Trends, Environment & markets), satellite map, My Tasks, PDF / CSV exports, decisions with an audit trail.
- **Department officer consoles** (25 departments): own-department view, approve → act → send completion report with photos → Collector verifies or returns.
- **Ask District IQ**: chat assistant in English, Tamil and Tanglish with charts, maps and tables; every number is checked against the data; officer answers are locked to their department.
- **AWS version**: collectors as Lambda functions behind API Gateway, data in S3 and DynamoDB, AI on Amazon Bedrock, scheduling by GitHub Actions (EventBridge is not allowed for the team role).

2. Problem statement and goal

A District Collector receives information from many disconnected channels: citizen grievance portals, police station reports, PWD flood and works registers, hospital MIS, IMD warnings, pollution boards and the daily news. Each comes in its own format, with its own IDs, and none of them talks to the others. The same waterlogged street may appear as a grievance, a police call, a PWD incident and a news story — four records, one problem.

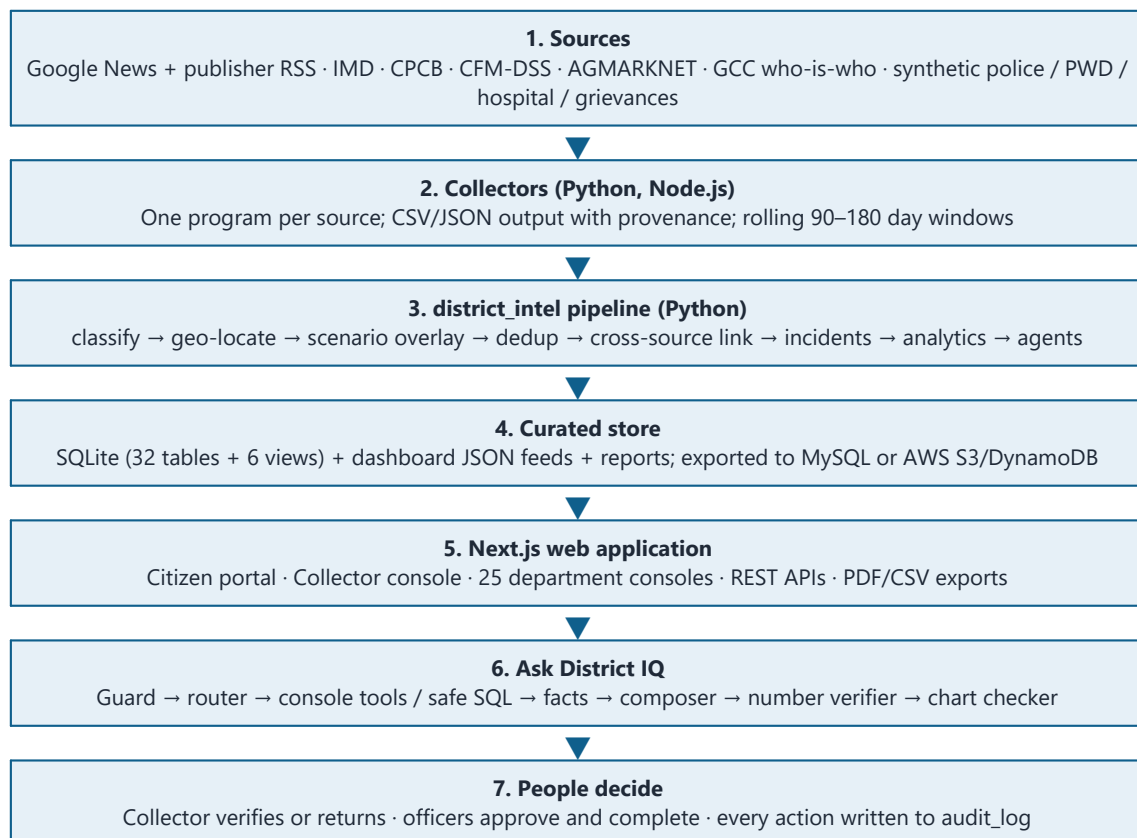
Goal: build a unified system that (1) collects all of these automatically, (2) turns them into one trustworthy picture of real incidents, (3) tells the Collector what needs attention today and why, (4) routes work to departments and tracks it to verified completion, and (5) answers ad-hoc questions in plain language — with every figure traceable to its source.

3. System architecture at a glance

The system is built in layers. Each layer has one job, and a lower layer never depends on an upper one. This lets collectors keep running even when the dashboard changes, and lets the dashboard be rebuilt without re-fetching any data.

Layer	Components	Responsibility
1. Collection	imd_weather_collector, cpcb_air_quality_collector, cfm_dss_collector, chennai_news_pipeline, police_dataset_generator, pwd_dataset_generator, chennai_hospital_data generator, grievance generator (portal)	Fetch or generate raw records into CSV/JSON files. Each owns exactly one domain.
2. Curation	district_intel (dintel package, run_pipeline.py)	Load, classify, locate, dedup, link, build incidents, analytics, agents, briefings, data-quality reports.
3. Storage	SQLite district_intel.db → MySQL (district_intel + district_intel_ops) → AWS S3 + DynamoDB	Pipeline-owned read store (replaced each build) and dashboard-owned ops store (decisions, never overwritten).
4. Application	Next.js 14 portal: /citizen, /collector, /officer, REST APIs under /api	Role-based consoles, maps, exports, decisions, the officer → Collector task pipeline.
5. Intelligence	Ask District IQ (src/lib/assistant, src/lib/ai), district_intel agents	Natural-language Q&A, briefings, gap finding, action planning — always number-verified.
6. Cloud	AWS Lambda, API Gateway, S3, DynamoDB, Bedrock, CloudWatch; GitHub Actions timer	Runs collectors and serves the store in the FarmwiseAI AWS account (ap-south-1).

Architecture flow



4. End-to-end workflow (data fetching to the Collector's decision)

This is the full journey of a single piece of information, for example heavy rain causing waterlogging in a ward.

#	Step	What happens	Where (code)
1	Schedule fires	Every day at 6:00 AM (Windows Task Scheduler DistrictIntel) or hourly / every 3 h (GitHub Actions → API Gateway → Lambda in the AWS version).	district_intel/scripts/run_intel.bat, .github/workflows/aws-refresh.yml
2	Collectors fetch	IMD rainfall + warnings, CFM-DSS river gauges and lake levels, CPCB AQI, news RSS (48 h window, first run 90-day backfill). Each source is retried 3 times; failures are logged, not hidden.	*_collector.py, chennai_news_pipeline/run_pipeline.py
3	Shared rain calendar	Police heavy-rain days + IMD warning days are passed to the synthetic generators through their own options so every generator agrees on the weather.	district_intel/dintel/sync.py
4	Generators refresh	Grievances, police, PWD and hospital records roll forward (180-day window).	*_generator, portal scripts
5	News processed	Cleaned, language detected, Chennai relevance filtered with a Tamil-aware gazetteer, tagged with 1 of 15 departments and a complaint flag.	chennai_news_pipeline/src
6	Load + normalise	8 loaders map every source into one 70-column events schema. Personal data such as mobile numbers is replaced by a keyed hash.	dintel/loaders/*
7	Classify	Crosswalk tables for structured sources; char-n-gram classifier for free text and "Other" complaints.	dintel/classify.py
8	Geo-locate	Point-in-polygon on 200 GCC ward polygons, snap ≤ 500 m (flagged), ward → zone → taluk.	dintel/geo.py, geolocate.py
9	Dedup	Union-find over blocked pairs; portal duplicate rule + officers' "Duplicate of" decisions.	dintel/dedup.py
10	Cross-source link	Blocking (category, KD-tree distance, time window) → logistic scorer → learned threshold → review band → union-find. One real incident = one row.	dintel/linking.py
11	Incidents	Lead record, status, deadline, priority score with reasons, attention flag (needs Collector).	dintel/incidents.py
12	Analytics	Poisson baselines with weekday factor, EWMA + z-score spikes, DBSCAN hotspots, Getis-Ord Gi*, KPIs.	dintel/analytics.py
13	Agents	Data Steward, Action Planner, Gap Finder, Linker, Watchdog, Briefing (with numeric verifier).	dintel/agents/*
14	Store + export	SQLite + CSV + JSON; then MySQL (atomic RENAME swap) or S3 content-addressed snapshot.	dintel/store.py, mysql_export.py, aws/export_intel.py
15	Console shows it	Collector sees the incident on the map, in Severity and My Tasks with plain-language reasons.	src/lib/collector/intel.ts, components/collector/app
16	Department acts	Officer approves → In action → Complete & send with remarks and site photo.	src/app/officer, src/lib/officer
17	Collector verifies	Verify or Send back with a note; decision + audit row written to the ops store.	/api/collector/decisions
18	Ask & report	Ask District IQ answers follow-up questions; PDF briefing / CSV action list exported.	src/lib/assistant, reportPdf.ts

5. Stage 1 — Data collection: every source and how it is fetched

A firm rule of the project: **collectors are never edited for downstream needs**. Each collector owns one domain, verifies its endpoints against live traffic, labels fallbacks clearly, and never bypasses CAPTCHAs or logins. Any data fix is done later in `district_intel`.

Source	Type	What it provides	How it is fetched	Notes / limits
News (<code>chennai_news_pipeline</code>)	Real	Chennai news in English and Tamil; ~20,600 articles	Google News RSS + publisher RSS (feedparser), article text via <code>trafilatura</code> , requests with retries	No API keys needed. 98% are headline snippets; full text only for some.
IMD weather	Real	Rainfall, temperature, humidity, 7-day forecast, district warnings with colour	IMD city-weather JSON API, GeoServer WFS district warnings, RMC Chennai fallbacks	4 stations (Nungambakkam, Meenambakkam, Madhavaram, Ennore). AWS/ARG portal needs login — not bypassed.
CPCB air quality	Real	AQI, category, prominent pollutant per CAAQMS station	<code>data.gov.in</code> API, else Playwright reads the public CCR map	History is CAPTCHA-gated, so it accumulates from our own runs. Needs a browser → runs on the PC.
CFM-DSS (TN WRD)	Real	River/canal gauge levels with warning/danger stages, lake storage, inflow, alerts, bulletins	<code>chennai.floodmonitor.tn.gov.in</code> JSON endpoints	No gauge history endpoint; 90-day history built by our runs.
AGMARKNET	Real	Vegetable and food prices at Chennai Uzhavar Sandhais, weekly change	AGMARKNET v1 open APIs with an honest User-Agent	Koyambedu doesn't report; daily report page is CAPTCHA-gated so not used.
GCC officials	Real	48 officials' names, roles, phone numbers	<code>chennaicorporation.gov.in</code> who-is-who (JS-rendered, read via browser)	Seeded into <code>official_contacts</code> .
The Hindu Chennai RSS	Real	Added source items on the map and briefing	RSS	Shown as violet pins.
Citizen grievances	Synthetic + real	Complaints with category, ward, map pin, status history; ~8,100 rows	Portal's generator script and real portal submissions	Uses the shared rain calendar.
Police incidents	Synthetic	16 taluks, 60 stations, ~12,300 incident reports	Node.js + TypeScript generator	Ground truth for dedup kept separately.
PWD	Synthetic	Flood incidents, works register, field records, lakes	Python generator	180 days.
Government hospitals	Synthetic	Bed occupancy, OPD, emergencies, disease cases, medicine stock, ambulances	Python generator	180 days.

6. Stage 2 — News pipeline (`chennai_news_pipeline`)

News is the only source that reports what the departments do not. The pipeline deliberately stays simple and honest: it fetches, cleans, filters for Chennai relevance, extracts neutral features and tags a department. It does not deduplicate, score, cluster or summarise — those are `district_intel`'s job, so each step can be tested on its own.

Step	What we did	Why
Fetch	Google News RSS queries + publisher RSS feeds; 90-day backfill on first run, then last 48 h.	No keys, stable, covers English and Tamil outlets.
Extract	<code>trafilatura</code> downloads the article page when allowed.	Better text than RSS summaries for classification.
Clean	HTML strip, Unicode normalisation, <code>langdetect</code> , date normalised to Asia/Kolkata (<code>tzdata</code>).	Tamil text and IST times shown correctly.
Relevance	Gazetteer of Chennai places in English and Tamil + exclusion rules.	Removes national news and other districts.
Classify	Keyword model → one of 15 departments + complaint flag.	Lets news be routed to the right department.
Store	Rolling 90-day <code>master_news.csv</code> , dated raw and processed files, optional MySQL.	Reprocess without re-fetching (<code>--reprocess</code>).

Tests: 88 pytest tests. HTTPS behind antivirus TLS inspection is handled with **truststore** (OS certificate store).

7. Stage 3 — district_intel: the curated intelligence layer

This is the heart of the project. It reads every collector's output and builds one curated, traceable, dashboard-ready store. A full build takes about 3.5 minutes on a laptop CPU.

Stage	Method used	Why this method
Load (8 loaders)	One loader per source into a 70-column canonical events table with provenance; personal data hashed.	Every downstream step works on one schema; we can always trace a row back to its file.
Category	Crosswalk YAML for structured sources; char-n-gram TF-IDF classifier trained on grievance, police and PWD text.	Char n-grams handle Tamil, Tanglish and spelling errors better than word models, and are tiny and fast.
News stories	One document per URL; Google copies merged into publisher copies; story clusters by char n-grams, then multilingual-e5-small embeddings (cosine 0.95, within 24 h).	Embeddings catch rephrased headlines; the strict threshold avoids false merges.
News incident filter	Classifier trained on weak labels + 300 hand labels, 5-fold cross-validated, population-weighted.	Separates real events (accidents, flooding) from notices, politics and court news.
Location	Point-in-polygon on 200 ward polygons (matplotlib.path), snap within 500 m, ward → taluk majority vote.	Real GCC geography instead of invented regions; no heavy GEOS dependency.
Scenario overlay	One shared world calendar; synthetic sources get the flood responses they would have produced on real rain days.	Separately generated datasets otherwise disagree about the weather; real sources are never changed. Every planted row is flagged.
Duplicates	Union-find over blocked candidate pairs; portal rule + officers' decisions; police and PWD rules.	Fast, transitive, explainable.
Cross-source linking	Blocking (category group, KD-tree, time window) → logistic scorer → threshold learned on half the events → gray review band → union-find.	Measured precision 0.99, F1 0.88. Logistic weights are explainable, unlike a black-box model.
Incidents	Lead record, status, deadline per category, priority computed at as-of time with reasons, attention flags.	The Collector sees why something is urgent.
Analytics	Poisson baselines with weekday factor, EWMA + z-scores, DBSCAN hotspots, Getis-Ord Gi*, KPIs by period and zone.	Standard, statistically sound methods; all numbers come from SQL/analytics, never from a language model.
Agents	Data Steward, Action Planner (playbook drafts), Gap Finder (news vs. records), Linker, Watchdog, Briefing with numeric verifier.	Rule-based by default; an optional LLM may rewrite text only if the verifier passes.
Outputs	district_intel.db, curated/*.csv (UTF-8 BOM for Tamil in Excel), dashboard/*.json, reports/*.md (evaluation, data quality, briefings).	Same data in the formats each consumer needs.

Main tables

Table	One row per
events	source record (complaint, police report, PWD incident, hospital alert, IMD warning, news incident)
incidents, incident_members, incident_timeline	real-world incident, its member records and merged timeline
actions	task (real PWD tasks + Action Planner drafts)
documents	news article / PWD announcement / CFM bulletin with story cluster
observations, observation_signals	metric reading / latest value (hospitals, lakes, IMD, CPCB, gauges)
kpis, daily_counts, anomalies, hotspots	KPI tile, count, spike, cluster
alerts, briefings, gaps, review_queue	agent outputs (drafts; people decide)
source_health, data_quality	Data Steward outputs
ref_*, world_calendar	reference masters (wards, taluks, departments, categories, facilities) and the shared day calendar

8. Stage 4 — Storage: SQLite, MySQL and AWS

Store	Owner	Contents	Why
SQLite district_intel.db	Pipeline	32 tables + 6 views, ~190 MB	Zero setup, single file, perfect for a batch build and tests.
MySQL district_intel	Pipeline	31 tables, ~370,000 rows; replaced each export via *__new tables + one atomic RENAME	Readers never see a half-loaded store.
MySQL district_intel_ops	Dashboard	collector_decisions, action_updates, audit_log, officer_reports, sources, mandi prices, assistant tables	Never dropped or truncated — it holds people's decisions.
AWS S3 (-raw, -uploads, -exports)	Both	Raw collector files, content-addressed intel snapshot (31 tables, 19.4 MB gzip), reference data	Cheap, durable, unchanged tables are skipped.
AWS DynamoDB (-intel, -portal, -summary, -user)	Both	Collector rows (diff-only writes), users, complaints, ops tables (58k rows migrated)	Serverless, pay per request, allowed by the challenge.
In-memory node:sqlite (website, AWS mode)	Website	Snapshot loaded from S3; MySQL SQL rewritten for SQLite by a dialect layer	Allowed services exclude RDS, so the app keeps its SQL without a DB server.

9. Stage 5 — The web application (chennai-grievance-portal-main)

9.1 Citizen portal

- Register / login with email OTP, password reset, profile; JWT in httpOnly cookies; role-protected routes (middleware).
- File a complaint: verified GCC complaint types, area → locality → street, ward from boundary polygons, Leaflet map pin, photo upload (tesseract.js OCR available), "Others" auto-classified by AI with a keyword fallback.
- Track complaint status and history; complaint codes; data export to CSV/Excel.

9.2 Collector console (/collector) — branded District IQ

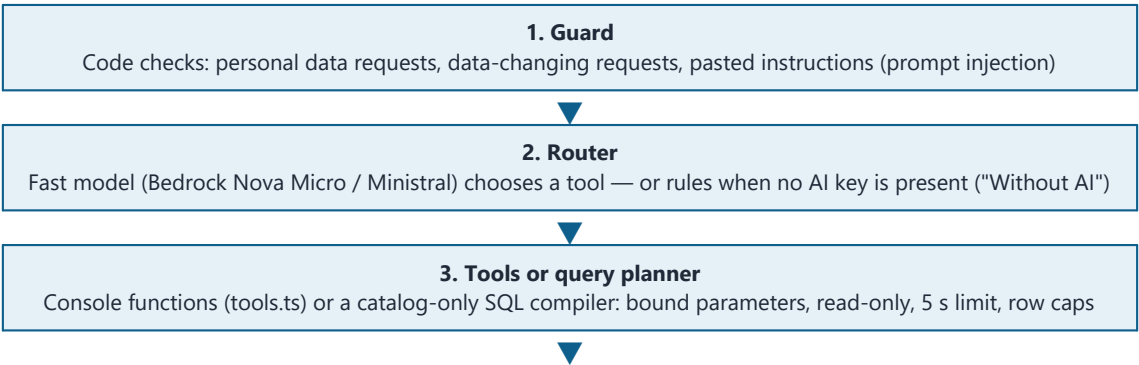
Page	What the Collector sees
1. Overview	Full-height satellite map (Leaflet + Esri imagery, ward/zone outlines, zoom & pan, violet pins for added sources); District Snapshot strip; By severity; My Tasks (incidents needing the Collector, officer completion reports first); Today's Briefing. Filters: period (Daily = today from midnight, Weekly, Monthly, Quarterly), department, zone, taluk.
2. Briefing	Readable briefing, developing stories (multi-article news threads), follow-ups, "Only in news" card (issues no department has recorded).
3. Trends	District-wide category rows, Monday-based weeks, spikes and hotspots; clicking opens a detail view (pattern()) instead of filtering.
4. Environment & markets	Rainfall, AQI and reservoir cards that follow the zone (nearest station named with distance), lakes, mandi prices per Chennai market.
Incident pop-up	Plain-language summary, facts, what needs attention, timeline, outlets counted by publisher, department completion report with photos.
Actions & exports	Verify / Send back decisions (audit logged); PDF report (jsPDF + autotable, top 10); CSV action list; top-bar chip showing feed freshness.

9.3 Department officer consoles (/officer) — 25 departments

- One account per department (officer.<code>@chennai.gov.in) with its own password; department read from the database on every request, never the URL.
- Page 1 Overview: the Collector's own overview filtered to the department — Severe events, Open incidents, Waiting for approval, Resolved, map, snapshot, My Work, briefing.
- Page 2 Work & insights: grievance board (New / In action / Sent to Collector / Verified), "What needs you now", complaint-type changes, and department-specific insight cards (police crime and response time, hospital beds and stock, PWD lakes and works, air quality for TNPCB, market prices for Revenue, rain for roads/electrical/parks/TANGEDCO).
- Task pipeline: New → Approve → In action → Complete & send (remarks + site photo) → Sent to Collector → Verified, or Returned with the Collector's note.
- Photos stored outside public/ and served only to the Collector and the sending department.

10. Stage 6 — Ask District IQ (AI assistant)

A pop-up (bottom-right launcher or Ctrl+K) on both consoles that answers questions in English, Tamil or Tanglish with charts (ECharts), maps and tables. **It never invents a number.**



4. Facts → Composer

The model writes the answer only from the returned facts



5. Number verifier + chart checker

Rejects any number not present in the data; validates the chart spec before display

- Follow-ups edit the previous answer ("only Zone 13", "make it a pie", "show on map"); pins re-run on fresh data; "Insights for today".
- Officer answers are locked to their department on the server (lockDept): rankings become own-department figures, other departments' incidents are dropped.
- Limits: 20 questions/min, 300/day per user, one in flight, CSV export $\leq 1,000$ rows (10/day). Spell correction, fuzzy matching, multilingual embeddings (transformers.js).
- Providers: Amazon Bedrock by default; Groq / OpenAI optional; npm run ai:key installs rotating AWS keys without a restart.

11. Stage 7 — AWS deployment and scheduling

The challenge asked that the platform run in the FarmwiseAI AWS account using only allowed services: S3, Lambda, API Gateway, DynamoDB, CloudWatch Logs and Bedrock (Nova Lite, Nova Micro, Ministral 3B/8B), region ap-south-1. MySQL was removed in the AWS version.

Piece	What we built	Why this way
setup.py	Re-runnable script that creates buckets, tables, Lambdas, the HTTP API and routes (names fai-tce-team37-*).	Repeatable infrastructure without IAM rights to use CloudFormation.
Collector Lambdas	One package (aws/lambda/collect) deployed 5 times with SOURCE env: imd, cpcb, cfm, pwd, hospital; plus news Lambda (~9 min, self-invoked async).	Reuses the unchanged collectors; diff-only DynamoDB writes keep costs low.
intel Lambda	/intel/upload-urls, /intel/publish, /intel/snapshot → presigned S3 links; manifest intel/current.json.	The heavy ML build (sentence-transformers) is too big for Lambda, so the PC builds and AWS stores.
store Lambda	/store/load and /store/write for the website's durable data.	The browser/website never holds AWS keys.
API Gateway (HTTP API)	POST /refresh/<src>, /ingest/<src> protected by x-refresh-key; throttled 1 rps, burst 5.	Single secure entry point; cheap.
Scheduling	GitHub Actions cron: hourly IMD/CFM/PWD/hospital; news every 3 h. PC task pushes CPCB (needs Playwright) and police (Node).	EventBridge is denied to the team role, so an external timer calls API Gateway.
Website data layer	src/lib/aws/store.ts loads the S3 snapshot into node:sqlite; dialect.ts rewrites MySQL SQL; DATA_BACKEND=aws switch.	No database server needed; same queries work in both versions.
Migration	aws/migrate_mysql.py moved 27 durable tables (58k rows) to DynamoDB and 17 reference tables to S3.	Idempotent; safe to re-run after expired tokens.
Bedrock	AI for the assistant via apac.amazon.nova-lite / nova-micro and Ministral 3B/8B.	Allowed models; data stays in AWS.

12. Complete functionality list

Module	Functionality
Data collection	8 automated feeds; retries; source health and freshness; rolling windows; backfill; reprocess without re-fetch; shared rain calendar.
Data quality	Data contract checks, quarantine, category drift, location validation (npm run validate:locations), evaluation report with measured accuracy.
Intelligence	Classification, geo-location, dedup, cross-source incidents, priority with reasons, deadlines, spikes, hotspots, gaps (news-only issues), developing stories, briefings (daily/weekly/monthly/quarterly).
Citizen	Sign-up with OTP, profile, file complaint with map + photo, AI category for "Others", track status.
Collector	4-page console, satellite map, filters, My Tasks, incident detail, verify/return decisions, PDF and CSV export, added sources, mandi prices, environment cards, feed status chip.
Department officer	Own-department console, approve and complete work with photos, returned-work handling, department insight cards, work record.
AI assistant	Multilingual Q&A, charts/maps/tables, follow-up edits, pins, insights for today, CSV export, verified numbers, department lock, offline rule mode.
Administration	Migrations, seeding (GCC reference, contacts, officers), per-department passwords, rate limits, audit log.
Cloud	Lambda collectors, API Gateway refresh, S3 snapshots, DynamoDB stores, Bedrock AI, GitHub Actions schedule.

13. Technology stack — what, where, why, and why better

Each technology was chosen for a specific job. The last column compares it with the obvious alternative.

Technology	Where we used it	What we did with it	Why we chose it	Why better than alternatives
Python 3.11	Collectors, news pipeline, district_intel, AWS scripts	All data fetching, cleaning, ML and analytics	Best data/ML ecosystem	vs Java/Node: far richer pandas/sklearn/geo libraries, faster to write
pandas / NumPy	district_intel, collectors	Load, join, aggregate every source	Fast columnar processing	vs plain loops/Excel: much faster and reproducible
scikit-learn	classify.py, linking.py, analytics.py	Char-n-gram classifier, logistic linker, DBSCAN, KD-tree	Proven, explainable models	vs deep learning: small data, CPU-only, explainable weights
SciPy	analytics.py	Poisson baselines, statistics	Correct statistical functions	vs hand-coded maths: tested and accurate
sentence-transformers (multilingual-e5-small)	embed.py (news stories)	Embeddings for English and Tamil headlines	Multilingual, small (470 MB), runs on CPU	vs OpenAI embeddings: free, offline, no data leaves the PC
matplotlib.path	geo.py	Point-in-polygon for 200 wards	No GEOS install needed	vs Shapely/GeoPandas: no native dependency trouble on Windows
feedparser, requests, trafilatura, langdetect	News pipeline	RSS fetch, HTTP, article text, language	Mature, key-free	vs paid news APIs: free, no quota, wider Tamil coverage
Playwright	CPCB collector	Reads the public CCR map	Handles JS-rendered pages	vs Selenium: faster, more reliable waits
truststore	All Python HTTP	Uses Windows certificate store	Works behind antivirus TLS inspection	vs disabling SSL verify: keeps HTTPS secure
PyYAML	config.yaml, reference/*.yaml	Tuning, categories, playbooks, deadlines	Human-readable config	vs hard-coded values: change behaviour without code
pytest / Vitest	All Python packages / portal	88 + 24 Python tests, 186 TypeScript tests, golden-question eval	Fast unit testing	vs manual testing: catches regressions each change
SQLite	district_intel.db; in-memory store in AWS mode	Build output, local reads	Zero-config single file	vs a server DB for batch builds: simpler, portable
MySQL 8 (mysql2)	Non-AWS version	Dashboard store and ops store	Concurrent reads/writes, familiar SQL	vs ORM: parameterised raw SQL is faster and transparent
Node.js + TypeScript	Police generator, portal	Type-safe server and generator code	One language front and back	vs plain JS: type errors caught at build time
Next.js 14 (App Router) + React 18	Portal, consoles, APIs	Pages + REST API routes in one app, middleware auth	Full-stack, server rendering	vs separate React + Express: one deploy, shared types
Tailwind CSS	Portal UI	Utility styling, responsive layout	Fast, consistent design	vs Bootstrap: lighter, fully custom look
Leaflet + OpenStreetMap / Esri imagery	Citizen map, Collector satellite map	Pins, ward outlines, zoom/pan	Free, no API key	vs Google Maps: no billing or key, open data
ECharts	Assistant charts	Bar, line, pie from verified facts	Rich, fast charts	vs Chart.js: more chart types, better large data
jsPDF + jspdf-autotable	Collector PDF report	Briefing PDF in the browser	Client-side, no server load	vs server PDF: instant, works offline
bcryptjs, jose (JWT), zod	Auth, validation	Password hashing, httpOnly JWT cookies, input schemas	Industry-standard security	vs sessions in localStorage: safer against XSS
nodemailer	OTP and reset email	Email delivery	Simple SMTP	vs paid services: no vendor lock-in
Vercel AI SDK (ai, @ai-sdk/*)	Assistant	Streaming answers, provider switching	One API for Bedrock/Groq/OpenAI	vs direct SDKs: switch model by config

Technology	Where we used it	What we did with it	Why we chose it	Why better than alternatives
@huggingface/transformers	Assistant embeddings	Multilingual matching in Node	Runs locally	vs hosted embeddings: free, private
Amazon Bedrock (Nova Lite/Micro, Ministral)	Assistant, AWS version	Routing and composing answers	Allowed by challenge, data stays in AWS	vs OpenAI: in-region, team-account billing
AWS Lambda	Collectors, intel, store	Serverless jobs	Pay per run, no servers	vs EC2: no idle cost (EC2 also not allowed)
Amazon API Gateway (HTTP API)	Refresh and data routes	Secure, throttled entry	Cheap HTTP API	vs public Lambda URLs: key check + throttling
Amazon S3	Raw data, snapshots, uploads	Content-addressed snapshots	Durable, cheap	vs DB blobs: far cheaper for large files
Amazon DynamoDB	Collector rows, users, ops	Key-value storage	Serverless, allowed	vs RDS: not allowed, and needs a running server
CloudWatch Logs	Lambda logs	Monitoring	Built-in	vs custom logging: zero setup
GitHub Actions	Schedule	Cron calls to API Gateway	Free timer	vs EventBridge: EventBridge denied to our role
Windows Task Scheduler	PC pipeline	Daily 6 AM district_intel run	Built into Windows	vs always-on server: no cost
Git + GitHub	Two repos	farmwise (MySQL) and farmwise-aws (AWS)	Version control, review	Keeps both versions cleanly separated

14. AI / ML and analytics methods

Method	Used for	Why	Measured result
Char n-gram TF-IDF + linear classifier	Complaint and news categories	Robust to Tamil/Tanglish and typos; fast	~0.99 on template text (expect lower on real text)
Weak labels + 300 hand labels, 5-fold CV	News incident filter	Better than rules alone	F1 0.63 (P 0.69, R 0.57), up from 0.41
multilingual-e5-small embeddings	News story clustering	Captures rephrased headlines	129 same-language merges checked by hand, all correct
Blocking + KD-tree + logistic scorer + union-find	Cross-source incident linking	Scales and explains	Precision 0.99, F1 0.88
Poisson baseline + weekday factor, EWMA, z-score	Spike detection	Counts data follows Poisson; robust to weekly cycles	Shown on Trends page
DBSCAN + Getis-Ord Gi*	Hotspots	Density clusters + statistical significance	Ward-level hotspot flags
Rule agents with numeric verifier	Briefings, gaps, action plans	No hallucinated numbers	Briefing numbers verified each build
LLM router + composer + verifier	Ask District IQ	Natural language with guaranteed-true figures	186 tests, golden-question eval set

15. Security, privacy and access control

- Passwords hashed with bcrypt; JWT in httpOnly cookies; middleware protects /citizen, /collector, /officer by role.
- Personal data (mobile numbers) replaced by keyed hashes in the intelligence store; assistant refuses personal-data requests.
- Officer department comes from the database on every request; other departments' records return 404.
- Parameterised SQL only; assistant SQL is catalog-only, read-only, time-limited, row-capped; optional SELECT-only DB user.
- Login rate limit 20/IP and 8/email per 15 min; assistant rate limits per user.
- AWS keys never in the browser; API Gateway key + throttling; per-department passwords in a git-ignored file.
- Collectors never bypass CAPTCHAs or logins; HTTPS always verified (truststore).

16. Quality measures, tests and known limits

Area	Status
Tests	district_intel 24 pytest, news pipeline 88 pytest, portal 186 Vitest, assistant golden questions (eval/), MySQL export row/column parity check.
Known limits	News filter misses ~4 in 10 incident articles; English–Tamil story merging disabled (not reliable from headlines); 98% of news are headline snippets; CPCB/CFM station coordinates approximate; rainfall/AQI history only accumulates from our own runs; synthetic grievance street names not tied to pins.
Data issues raised	Power-cut news sometimes filed under GCC instead of TANGEDCO; CMWSSB, Police and PWD have no GCC contacts; Kolathur taluk has no ward list.
AWS gaps	No EventBridge (external cron used); ML build too big for Lambda (PC builds, AWS stores); AWS keys rotate every few hours.

17. Development timeline

Date (2026)	Milestone
26 Sep	Initial commit of all data projects; Task 6 blueprint agreed (real ward geography, cross-source linking, SQL for numbers, agents on top).
27 Sep	district_intel data layer, shared rain calendar, hand-labelled news filter, 180-day data; MySQL export; first Collector dashboard.

Date (2026)	Milestone
28 Sep	Round 1–2: District IQ brand, fixed-screen console, satellite map, PDF report, GCC contacts, news routing.
29 Sep	Rounds 3–6: 4-page console, added sources, AGMARKNET mandi prices, developing stories, taluk filter, location checks, daily 6 AM data.
30 Sep	Rounds 7–8: map layout, Trends without filters, explainable page 1, action-list export, data checks.
1 Oct	Rounds 9–14: Daily = today; officer consoles merged and linked to the Collector; Ask District IQ; Work & insights page; AWS backend (S3, DynamoDB, Lambdas, API Gateway, Bedrock); repo split.

18. How to run

Task	Command
Build the intelligence store	<code>cd district_intel; python run_pipeline.py build</code>
Refresh due sources	<code>python run_pipeline.py refresh</code>
Export to MySQL	<code>python run_pipeline.py mysql</code>
Run the news pipeline	<code>cd chennai_news_pipeline; python run_pipeline.py</code>
Portal setup	<code>npm install; npm run db:setup; npm run setup:ops; npm run setup:officer; npm run seed:officers</code>
Start the web app	<code>npm run dev (http://localhost:3000)</code>
Install AI key	<code>npm run ai:key</code>
Tests	<code>python -m pytest -q tests; npx vitest run</code>
AWS deploy / export	<code>python aws/setup.py; python aws/export_intel.py; python aws/push.py cpcb police</code>